

Access Governance Engine

A ServiceNow scoped application for automated employee offboarding and access revocation

Technical Build Report

Dhyanam Joshi

ServiceNow Administrator & Developer | Sydney, Australia

September 2026

Table of Contents

What This Application Does.....	2
1. Architecture Overview.....	2
2. Data Model.....	3
3. Security Context Lookup.....	4
4. The Request Form (Record Producer).....	5
5. Access Control.....	6
6. The Automation Engine.....	10
7. Proof It Works: End to End.....	16

What This Application Does

When someone leaves a company, or changes teams, or just needs a security review, someone has to go and manually check every role and every group that person holds, then remove the ones that no longer apply. Done by hand, that's slow, easy to get wrong, and leaves no real record of who decided what. This application replaces that manual process with a governed workflow: a manager or an HR person submits a request through the Service Catalog, a security reviewer approves it, and the platform itself carries out the actual access changes, automatically and safely, with a full audit trail of exactly what was removed, who asked for it, and who signed off.

There are three governance actions the application supports: Review (just show what a user currently has, no changes made), Revoke groups (remove specific group memberships the reviewer selects), and Offboard user (remove everything: every group and every remaining direct role, and deactivate the account). Each one maps to a real HR/IT scenario, and each one is backed by logic that was tested directly against the platform's real behavior rather than assumed.

1. Architecture Overview

The application is built as three separate pieces of server-side logic, each with exactly one job, rather than one large script that tries to do everything. This separation wasn't arbitrary. It came directly out of a real platform constraint discovered while building the revoke logic: scoped applications in ServiceNow are not allowed to delete rows from the security tables that control roles and group memberships (`sys_user_has_role` and `sys_user_grmember`), even when every other permission has been granted. That's a deliberate table-level protection, confirmed by directly attempting the delete and watching the platform refuse it.

The fix is a pattern sometimes called a trusted intermediary: a small, narrowly-scoped piece of code sits in the Global application scope, which does have the platform-level trust to perform these deletes, and everything else calls into it only when it genuinely needs to. The rest of the application's logic, including all the decision-making about what should be revoked and all the audit logging, stays in the app's own scope, deliberately kept away from that privileged capability.

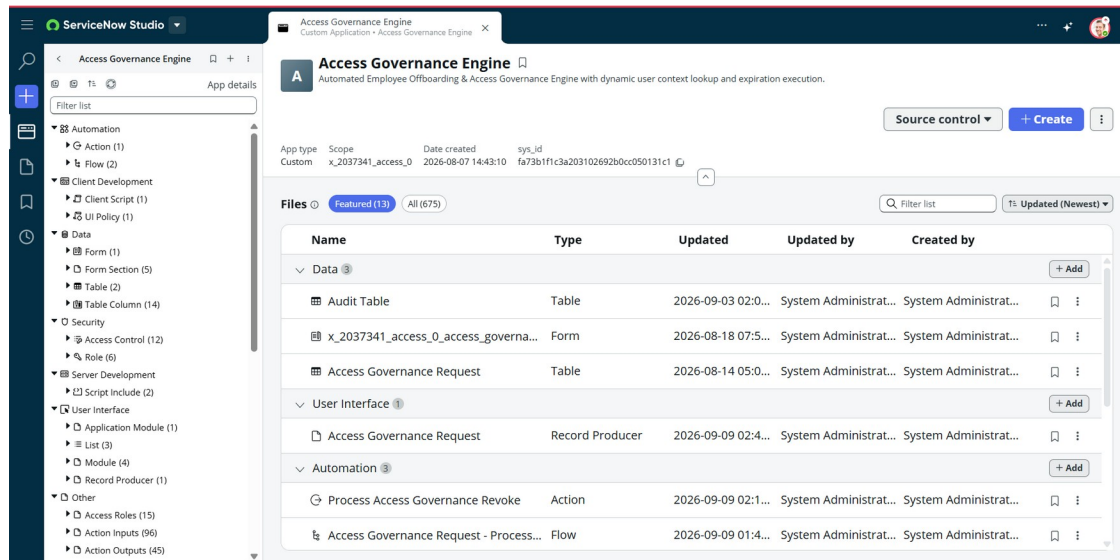


Figure 1: The application's file structure inside Studio: tables, roles, ACLs, the Record Producer, the Flow, and the Script Includes, all organized under one scoped application.

Name	Application	Active	Description	Updated
AccessGovernanceGlobalHelper	Global	true		2026-09-09 02:19:33
AccessGovernanceRevokeProcessor	Access Governance Engine	true	Recieves Request from flow designer afte...	2026-09-08 08:15:42
AccessGovernanceUtils	Access Governance Engine	true	Fetches active roles and group membershi...	2026-08-17 05:51:46

Figure 2: The three Script Includes side by side. Notice AccessGovernanceGlobalHelper lives in Global scope while the other two live in the application's own scope; that split is the whole point of the trusted-intermediary design.

The three components

- AccessGovernanceUtils (app scope, client-callable): read-only. Given a user, it looks up their current roles and group memberships and hands that back to the form. It never changes anything.
- AccessGovernanceRevokeProcessor (app scope, server-only): the decision-maker. Given a request's target user, the action chosen, and who requested/approved it, it figures out what actually needs to happen, writes the audit trail, and calls the Global helper to carry out the deletion.
- AccessGovernanceGlobalHelper (Global scope, server-only, not client-callable): the only piece of code in the entire application that actually deletes a role or group assignment. Kept deliberately small and narrow: it doesn't decide anything, it just executes.

Calling from the app's own scope into a Script Include sitting in Global scope turned out to need three separate things to work, not one, each one discovered by hitting a real error and tracing it down rather than guessing:

- A cross-scope privilege record, which the platform actually generates for you the first time the attempt is made, but that first attempt is still refused, so the real call only succeeds on the second try.
- The target Script Include's 'Accessible from' setting has to explicitly say 'All application scopes'; without it, the class is effectively invisible to anything outside its own scope, no matter what privileges exist.
- The calling code has to use an explicit global. prefix when creating the object (new global.AccessGovernanceGlobalHelper()), confirmed by the platform's own error message, which literally says a global qualifier is missing.

2. Data Model

Two custom tables hold everything the application needs. The request table extends Task, which means it gets state, assigned_to, opened_by, and work_notes for free: that's what actually powers the approval and fulfillment workflow, since a request needs to move through states and be assigned to whoever's handling it, and building that from scratch would have meant reinventing something the platform already does well.

Column label	Type	Reference	Max length	Default value	Display	Updated
Revoke Groups	List	Group Member	1,024		false	2026-09-02 07:17:30
Target User	Reference	User	32		false	2026-08-14 05:21:33
Governance Action	Choice	(empty)	40		false	2026-08-14 05:21:32
Security Context	String	(empty)	4,000		false	2026-08-14 05:21:31

Figure 3: The Access Governance Request table's own fields: who the request concerns, which groups to revoke, what action was chosen, and a frozen snapshot of the user's access at request time.

Column label	Type	Reference	Max length	Default value	Display	Updated
Requested By	Reference	User	32		false	2026-09-06 00:20:19
Target User	Reference	User	32		false	2026-09-03 02:14:37
Revoke Type	Choice	(empty)	40		false	2026-09-03 02:14:37
Revoke Name	String	(empty)	400		false	2026-09-03 02:14:36
Date	Date/Time	(empty)	40		false	2026-09-03 02:14:35
Approved By	Reference	User	32		false	2026-09-03 02:14:35
Access Governance Id	Reference	Access Governance Request	32		false	2026-09-03 02:14:34
Short Description	String	(empty)	400		false	2026-09-03 02:14:34
Updates	Integer	(empty)	40		false	2026-09-03 02:03:30
Updated	Date/Time	(empty)	40		false	2026-09-03 02:03:30
Created by	String	(empty)	40		false	2026-09-03 02:03:30
Updated by	String	(empty)	40		false	2026-09-03 02:03:30
Created	Date/Time	(empty)	40		false	2026-09-03 02:03:30
Sys ID	Sys ID (GUID)	(empty)	32		false	2026-09-03 02:03:29

Figure 4: The Audit Table's fields. Every one of these gets filled in before a deletion happens, not after, more on why below.

Why the Governance Action choice list matters more than it looks

Each governance action has both a display label a person sees (Review, Revoke, Offboard) and a separate internal value the platform actually stores (review_roles_groups, revoke_roles_groups, offboard_user). That distinction caused a real bug later, in the Flow automation: the branching logic was originally written to check against the display labels, and it worked fine in every manual test, right up until real data from an actual submitted record flowed through, at which point the internal value showed up instead of the label, and the logic silently fell into its 'unrecognized action' branch. It's a good example of why testing with hand-typed values isn't the same as testing with the real thing.

Label	Value	Language	Sequence	Inactive	Updated
Offboard	offboard_user	en	2	false	2026-09-02 07:13:54
Review	review_roles_groups	en	0	false	2026-09-02 07:13:47
Revoke	revoke_roles_groups	en	1	false	2026-09-02 07:12:59
+ Insert a new row...					

Figure 5: The Governance Action choice list, showing the display label and the actual stored internal value side by side.

3. Security Context Lookup

The first thing a reviewer needs, before deciding anything, is a clear picture of what the target user currently has. As soon as a user is selected on the request form, a small piece of client-side script calls AccessGovernanceUtils in the background and populates a read-only text block with that user's current roles and group memberships, with no page reload and no separate step.

Service Catalog > Office > Services > Access Governance Request

Request access review, group revocation, or full offboarding for a user

Use this form to request a security review, revoke specific group access, or fully offboard a departing employee. Requests are routed to the security team for approval and fulfillment.

* Target User
Abel Tuter

* Governance Action
Review

Current Security Context

--- ACTIVE SECURITY CONTEXT ---
 ROLES: interaction_agent, cmdb_ms_user, sn_request_write, template_editor, view_changer, cmdb_query_builder_read, agent_workspace_user, sn_change_write, email_client_template_read, viz_creator, sn_request_approver_read, sn_sow_it_agent_dashboard_user, snic_required_script_writer_permission, sn_incident_write, sn_gd_guidance.guidance_user, certification, sn_sttrm_condition_read, sn_sow.sow_home, data_manager_user, sn_sow.sow_user, sn_item_recomm.recommendations_read, service_status_subscriber, dependency_views, tracked_file_reader, sn_change_read, sn_incident_read, cmdb_read, email_composer, sn_itsm_contact_query, sn_uib_collab.user, sn_request_read, workspace_user, sn_problem_write, app_service_user, canvas_user, sn_cmdb_user, sn_problem_read, template_read_global, task_editor, sn_nb_action.next_best_action_user, sn_sow.sow_list, itil, sn_sla_definition_query, cmdb_query_builder, snc_platform_rest_api_access

GROUPS: testGroup, Conditional Script Writer

Submit

Figure 6: A real request form, mid-fill: Abel Tuter selected as the target user, and the Current Security Context box already populated with his actual roles and groups, pulled live from the platform.

4. The Request Form (Record Producer)

The request form lives in the Service Catalog, the same place an employee would go to request a laptop or new software, which matters because it means HR and managers never need direct access to the underlying table or any admin screen. They fill out a simple form and submit it.

Service Catalog > Office > Services > Access Governance Request

Request access review, group revocation, or full offboarding for a user

Use this form to request a security review, revoke specific group access, or fully offboard a departing employee. Requests are routed to the security team for approval and fulfillment.

* Target User
Abel Tuter

* Governance Action
Revoke

Groups to Revoke

Add Filter Run filter

-- choose field -- -- oper -- -- value --

Available

Conditional Script Writer
testGroup

Selected

--None--

Current Security Context

--- ACTIVE SECURITY CONTEXT ---
 ROLES: interaction_agent, cmdb_ms_user, sn_request_write, template_editor, view_changer, cmdb_query_builder_read, agent_workspace_user, sn_change_write, email_client_template_read, viz_creator, sn_request_approver_read, sn_sow_it_agent_dashboard_user, snic_required_script_writer_permission, sn_incident_write, sn_gd_guidance.guidance_user, certification, sn_sttrm_condition_read, sn_sow.sow_home, data_manager_user, sn_sow.sow_user, sn_item_recomm.recommendations_read, service_status_subscriber, dependency_views, tracked_file_reader, sn_change_read, sn_incident_read, cmdb_read, email_composer, sn_itsm_contact_query, sn_uib_collab.user, sn_request_read, workspace_user, sn_problem_write, app_service_user, canvas_user, sn_cmdb_user, sn_problem_read, template_read_global, task_editor, sn_nb_action.next_best_action_user, sn_sow.sow_list, itil, sn_sla_definition_query, cmdb_query_builder, snc_platform_rest_api_access

GROUPS: testGroup, Conditional Script Writer

Figure 7: The live form with Revoke selected. Notice the Groups to Revoke picker on the left only shows testGroup and Conditional Script Writer, the two groups this specific user actually belongs to, not every group in the system.

That filtering wasn't automatic: it's a real feature, added after testing the form myself and realizing that showing every group in the company to pick from was going to be confusing and error-prone in practice, exactly the kind of thing a real user would hit on their very first try. It's built as a reference qualifier: a small script attached to the Groups to Revoke field that queries the target user's actual memberships and returns only those as valid options, re-evaluating automatically every time a different user is selected.

Figure 8: The Catalog UI Policy that shows or hides the Groups to Revoke field depending on which action is selected. 'Reverse if false' means this one rule covers both directions (show it for Revoke, hide it for everything else) without needing a separate rule for each case.

5. Access Control

Every table and every entry point has a specific, deliberately narrow role attached to it, rather than relying on broad defaults. The guiding idea throughout was separation of duties: the person requesting a change should never be the same person who approves it, and the people who can see compliance history shouldn't automatically be the same people who can act on live requests.

The audit table: write access locked to admin, read access to a dedicated role

Figure 9: Create on the audit table requires admin. No ordinary user, requester, or approver can manually insert a row.

Access Control
x_2037341_access_0_audit_table

* Type: record

* Operation: delete

Admin overrides: ☒

Decision Type: Allow If

Protection policy: -- None --

* Name: Audit Table [x_2037341_access_0_audit_table] -- None --

Description: Allow delete for records in x_2037341_access_0_audit_table, for users with role admin.

Application: Access Governance Engine

Active: ☒

Advanced: ☐

Conditions

Requires role

Role
admin

Figure 10: Delete requires admin too. Audit history shouldn't be something a normal workflow can erase.

Access Control
x_2037341_access_0_audit_table

* Type: record

* Operation: write

Admin overrides: ☒

Decision Type: Allow If

Protection policy: -- None --

* Name: Audit Table [x_2037341_access_0_audit_table] -- None --

Description: Allow write for records in x_2037341_access_0_audit_table, for users with role admin.

Application: Access Governance Engine

Active: ☒

Advanced: ☐

Conditions

Requires role

Role
admin

Figure 11: Write requires admin as well, for the same reason.

Access Control
x_2037341_access_0_audit_table

* Type: record

* Operation: read

Application: Access Governance Engine

Active: ☒

Advanced: ☐

Admin overrides: ☒

Decision Type: Allow If

Protection policy: -- None --

* Name: Audit Table [x_2037341_access_0_audit_table]

Description: Allow read for records in x_2037341_access_0_audit_table, for users with role x_2037341_access_0.audit_table_user.

Conditions

Requires role

Role
x_2037341_access_0.audit_table_user

Figure 12: Read, on the other hand, only requires a dedicated audit_table_user role, deliberately not admin, since 'can review compliance history' shouldn't require full administrative rights.

One thing worth being upfront about: the application's own audit-writing code still works despite the admin-only write restriction, because plain GlideRecord operations in server-side scripts bypass ACL enforcement in ServiceNow, confirmed against ServiceNow's own community documentation before relying on it. So the restriction genuinely blocks a person from manually touching the table, while the application's legitimate, automated writes go through exactly as intended.

The request table: approval requires its own role

Access Control
x_2037341_access_0_access_governance_request

* Type: record

* Operation: write

Application: Access Governance Engine

Active: ☒

Advanced: ☐

Admin overrides: ☒

Decision Type: Allow If

Protection policy: -- None --

* Name: Access Governance Request [x_2037341_access_0_acces...]

Description: Allow write for records in x_2037341_access_0_access_governance_request, for users with role x_2037341_access_0.approver.

Conditions

Access Control Rules have two decision types, and these types will behave differently depending on conditions.

1. Allow Access: Allows access to a resource if all conditions are met. Additional ACLs may grant access to records where this ACL has not granted access.
2. Deny Access: Denies access to a resource unless all conditions are met. Additional ACLs may not grant access to records where this ACL has denied access.

[More Info](#)

Requires role

Role
x_2037341_access_0.approver

Figure 13: Write access on the request table requires a dedicated approver role. Originally this was left on the application's generic default role, which on reflection meant any user of the app, including the original requester, could have approved their own request. Narrowing it down closed that gap.

Roles, laid out clearly

☐	Name	Description	Elevated privilege	Updated
	x_2037341_access_0.approver	Allows an ITIL/security user to approve and fulfill Access Governance Requests	false	2026-09-09 03:21:12
	x_2037341_access_0.hr_requester	Allows an HR team member to submit Offboard user requests via the Access Governance Request catalog item	false	2026-09-09 02:37:40
	x_2037341_access_0.manager_requester	Allows a manager to submit Revoke groups requests via the Access Governance Request catalog item	false	2026-09-09 02:37:10
	x_2037341_access_0.audit_table_user		false	2026-09-03 02:03:02
	x_2037341_access_0.user	Default user role	false	2026-08-07 07:43:11
	x_2037341_access_0.admin	Default admin role	false	2026-08-07 07:43:10

Figure 14: Every custom role the application defines, each with one clear purpose: two roles gate who can submit a request (by scenario: a manager revoking access versus HR offboarding someone), a separate role gates who can approve one, and a separate role again gates who can read audit history.

Gating the request form itself

Being able to write to the table once a request exists is only half the picture: the form to create one in the first place also needs to be restricted, otherwise anyone browsing the Service Catalog could submit a request regardless of role. That's handled through a User Criteria record, ServiceNow's standard mechanism for controlling who can even see a catalog item, tied to the two requester roles.

Record Producer

Access Governance Request

Copy

Try It

Update

Edit in Catalog Builder

Delete

1 /** This script is executed before the Record is generated

2 * `current` - *GlideRecord* produced by Record Producer

3 * Don't use `current.update()` or `current.insert()` as the record is generated by Record Producer

4 * Don't use `current.setValue('sys\_class\_name', 'xxx')` as this will trigger reparent flow and can cause data loss

5 * Avoid `current.setAbortAction()` and generate a separate record

6 * Use `producer.var1` to access variables

7 */

Images (desktop view)

Picture Click to add...

Icon Click to add...

Copy

Try It

Update

Edit in Catalog Builder

Delete

Related Links

[Item Diagnostic](#)

[Run Point Scan](#)

Variables (4)

Variable Sets

Catalog UI Policies (1)

Catalog Client Scripts (1)

Available For (1)

Not Available For

Categories (1)

Catalogs (1)

Catalog Data Lookup Definitions

Related Articles

Related Catalog Items

Assigned Topics

for text

Search

Actions on selected rows...

New

Edit...

Catalog Item = Access Governance Request

Available for

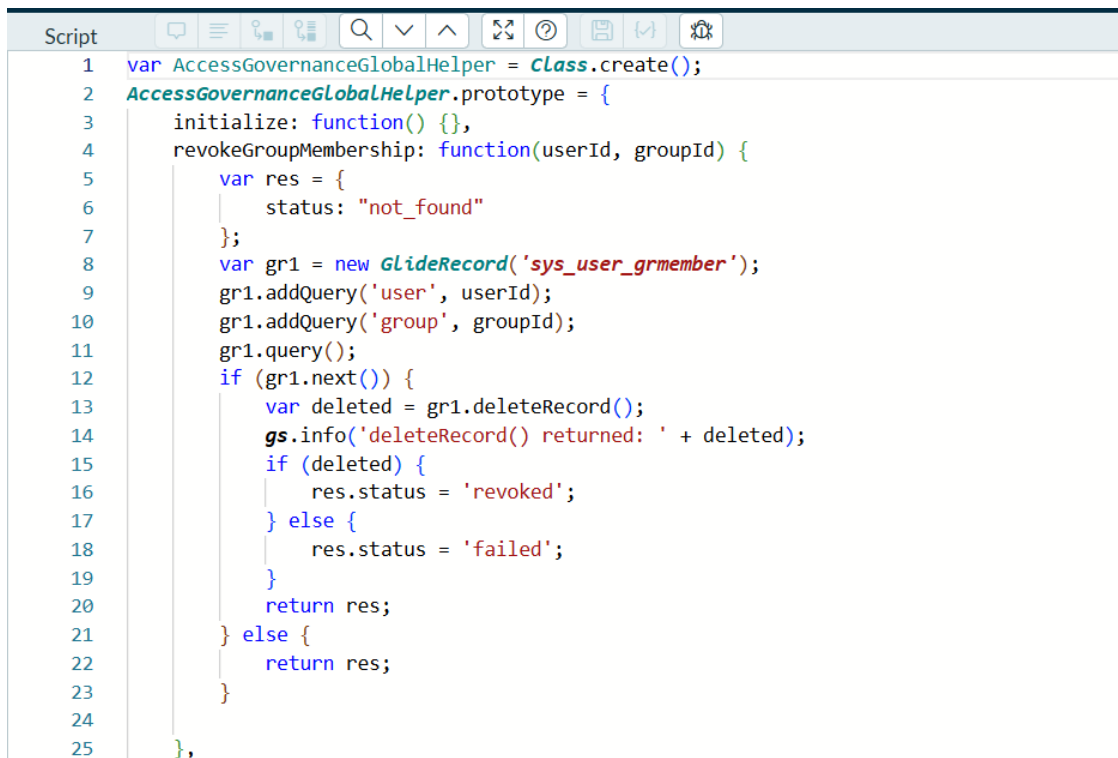
Access Governance Requesters

Figure 15: The Record Producer's Available For setting, pointing at a User Criteria record scoped to exactly the two requester roles. Anyone without one of those roles won't see this item in the catalog at all.

6. The Automation Engine

Everything up to this point (the request form, the security context lookup, the access controls) exists to feed one final piece: the moment an approver marks a request Closed Complete, the actual access change needs to happen by itself, with nobody running a script by hand. Getting that one connection working turned out to be the hardest part of the entire build, and it's worth walking through honestly, because the bugs along the way taught me more about how ServiceNow's platform actually behaves than anything else in this project.

Here's the actual code behind that final step: the privileged Global-scope helper that performs every delete, and the app-scoped processor that decides what to do and writes the audit trail before ever calling it.



```
Script
1  var AccessGovernanceGlobalHelper = Class.create();
2  AccessGovernanceGlobalHelper.prototype = {
3      initialize: function() {},
4      revokeGroupMembership: function(userId, groupId) {
5          var res = {
6              status: "not_found"
7          };
8          var gr1 = new GlideRecord('sys_user_grmember');
9          gr1.addQuery('user', userId);
10         gr1.addQuery('group', groupId);
11         gr1.query();
12         if (gr1.next()) {
13             var deleted = gr1.deleteRecord();
14             gs.info('deleteRecord() returned: ' + deleted);
15             if (deleted) {
16                 res.status = 'revoked';
17             } else {
18                 res.status = 'failed';
19             }
20             return res;
21         } else {
22             return res;
23         }
24     },
25 }
```

Figure 16: AccessGovernanceGlobalHelper, part one: initialize() and revokeGroupMembership(), the only method in the entire application that actually deletes a sys_user_grmember row, returning a three-state result of revoked, failed, or not_found.

```

Script
1  var AccessGovernanceGlobalHelper = Class.create();
26  offboardUser: function(userId) {
27      var res = {
28          status: 'success',
29          groupsRemoved: 0,
30          directRolesRemoved: 0,
31          accountDeactivated: false
32      };
33      //Piece A: Find all groups , revoke each one
34      var groupGr = new GlideRecord('sys_user_grmember');
35      groupGr.addQuery('user', userId);
36      groupGr.query();
37      while (groupGr.next()) {
38          var groupResult = this.revokeGroupMembership(userId, groupGr.group.toString());
39          if (groupResult.status === 'revoked') {
40              res.groupsRemoved++;
41          }
42      }
43      //Piece B Removing direct roles where inherited=false
44      var roleGr = new GlideRecord('sys_user_has_role');
45      roleGr.addQuery('user', userId);
46      roleGr.addQuery('inherited', false);
47      roleGr.query();
48      var directRoleCount = roleGr.getRowCount();
49      roleGr.deleteMultiple();
50      res.directRolesRemoved = directRoleCount;
51
52      // Piece C: Deactivate the account
53      var userGr = new GlideRecord('sys_user');
54      userGr.addQuery('sys_id', userId);
55      userGr.query();
56      if (userGr.next()) {
57          userGr.active = false;
58          var updateResult = userGr.update();
59          res.accountDeactivated = updateResult ? true : false;
60      }
61      return res;
62  },
63  },
64
65  type: 'AccessGovernanceGlobalHelper'
66  };

```

Figure 17: AccessGovernanceGlobalHelper, part two: offboardUser(), showing Piece A (looping through every group and revoking each one by calling this.revokeGroupMembership()), Piece B (deleting the remaining inherited:false roles with deleteMultiple(), counted first with getRowCount() since deleteMultiple() itself doesn't report a count), and Piece C (deactivating the account and checking the real return value of update()).

```

Script
1  var AccessGovernanceRevokeProcessor = Class.create();
2  AccessGovernanceRevokeProcessor.prototype = {
3      initialize: function() {},
4      processRequest: function(userId, action, groupsList, requestedBy, approvedBy, accessGovernanceId) {
5          var res = {
6              status: 'success',
7              action: action,
8              result: ''
9          };
10         // your branching logic here
11         if (action == 'offboard_user') {
12             var user = new GlideRecord('sys_user');
13             user.addQuery('sys_id', userId);
14             user.query();
15             if (user.next()) {
16                 var username = user.name;
17             }
18             var gr1 = new GlideRecord('sys_user_grmember');
19             gr1.addQuery('user', userId);
20             gr1.query();
21             while (gr1.next()) {
22                 var grname = gr1.group.getDisplayValue();
23                 var auditGr1 = new GlideRecord('x_2037341_access_0_audit_table'); //Creating GlideRecord Query to insert the auditing details into the audit table
24                 auditGr1.initialize();
25                 auditGr1.target_user = userId;
26                 auditGr1.revoke_type = 'Group';
27                 auditGr1.revoke_name = grname;
28                 auditGr1.requested_by = requestedBy;
29                 auditGr1.approved_by = approvedBy;
30                 auditGr1.access_governance_id = accessGovernanceId;
31                 auditGr1.short_description = 'Revoked group ' + grname + ' from ' + username;
32                 auditGr1.date = new GlideDateTime();
33                 auditGr1.insert();
34             }

```

Figure 18: AccessGovernanceRevokeProcessor, part one: the top of processRequest(), showing the offboard_user branch begin: querying the user's real groups and writing one audit row per group before anything is actually deleted.

```

35     var userHasRole = new GlideRecord('sys_user_has_role');
36     userHasRole.addQuery('user', userId);
37     userHasRole.addQuery('Inherited', false);
38     userHasRole.query();
39     while (userHasRole.next()) {
40         var rolename = userHasRole.role.getDisplayValue();
41         var auditGr2 = new GlideRecord('x_2037341_access_0_audit_table'); //Creating GlideRecord Query to insert the auditing details into the audit table
42         auditGr2.initialize();
43         auditGr2.target_user = userId;
44         auditGr2.revoke_type = 'Role';
45         auditGr2.revoke_name = rolename;
46         auditGr2.requested_by = requestedBy;
47         auditGr2.approved_by = approvedBy;
48         auditGr2.access_governance_id = accessGovernanceId;
49         auditGr2.short_description = 'Revoked role ' + rolename + ' from ' + username;
50         auditGr2.date = new GlideDateTime();
51         auditGr2.insert();
52     }
53
54
55     var offboard = new global.AccessGovernanceGlobalHelper();
56     var r = offboard.offboardUser(userId);
57     res.result = r;
58 } else if (action == 'revoke_roles_groups') {
59     var user = new GlideRecord('sys_user');
60     user.addQuery('sys_id', userId);
61     user.query();
62     if (user.next()) {
63         var username = user.name;
64     }
65     var revoke = new global.AccessGovernanceGlobalHelper();
66     var groupArray = groupsList.split(',');
67     var resultarr = [];
68     for (var i = 0; i < groupArray.length; i++) {
69         var groupGr = new GlideRecord('sys_user_group');
70         groupGr.addQuery('sys_id', groupArray[i]);
71         groupGr.query();
72         if (groupGr.next()) {
73             var groupname = groupGr.name;
74         } else {
75             groupname = 'Unknown Group';
76         }

```

Figure 19: AccessGovernanceRevokeProcessor, part two: the offboard_user branch finishing (the same audit-before-delete pattern applied to the user's direct roles), followed by the start of the revoke_roles_groups branch splitting the incoming group list back into individual group IDs.

```

1     var AccessGovernanceRevokeProcessor = Class.create();
4     processRequest: function(userId, action, groupsList, requestedBy, approvedBy, accessGovernanceId) {
66         var groupArray = groupsList.split(',');
67         var resultarr = [];
68         for (var i = 0; i < groupArray.length; i++) {
69             var groupGr = new GlideRecord('sys_user_group');
70             groupGr.addQuery('sys_id', groupArray[i]);
71             groupGr.query();
72             if (groupGr.next()) {
73                 var groupname = groupGr.name;
74             } else {
75                 groupname = 'Unknown Group';
76             }
77             var auditGr = new GlideRecord('x_2037341_access_0_audit_table'); //Creating GlideRecord Query to insert the auditing details into the audit table
78             auditGr.initialize();
79             auditGr.target_user = userId;
80             auditGr.revoke_type = 'Group';
81             auditGr.revoke_name = groupname;
82             auditGr.requested_by = requestedBy;
83             auditGr.approved_by = approvedBy;
84             auditGr.access_governance_id = accessGovernanceId;
85             auditGr.short_description = 'Revoked group ' + groupname + ' from ' + username;
86             auditGr.date = new GlideDateTime();
87             auditGr.insert();
88             var rr = revoke.revokeGroupMembership(userId, groupArray[i]);
89             resultarr.push(rr);
90         }
91         res.results = resultarr;
92
93
94
95     } else if (action == 'review_roles_groups') {
96         // No automated action for Review - security context already
97         // captured at request-creation time via AccessGovernanceUtils
98
99     } else {
100         res.status = 'error';
101         res.message = 'Unrecognized action: ' + action;
102     }
103     return res;
104 },
105
106 type: 'AccessGovernanceRevokeProcessor'
107 };

```

Figure 20: AccessGovernanceRevokeProcessor, part three: the revoke_roles_groups branch completing (one audit row per group, then the real call to revoke.revokeGroupMembership()), the review_roles_groups branch (intentionally a no-op, since the security context was already captured at request time), and the fallback branch for an action value the code doesn't recognize.

Getting from a state change to a Script Include. Flow Designer doesn't have a built-in action for calling a Script Include directly. There's no "run script" block sitting in the standard action list. The documented pattern is to build a separate Custom Action in Action Designer, put a Script step inside it, and then call that Action from the Flow like any other built-in step. So the Flow itself is short: it triggers on Access Governance Request records being Updated, with a condition that the State field is Closed Complete, set to run only once per record so it

doesn't re-fire every time someone re-saves an already-closed request. From there it hands off to a Custom Action called Process Access Governance Revoke, which is where the real logic lives.



Figure 21: The finished Flow: a trigger on State = Closed Complete, a Look Up Record step that re-fetches the same record fresh by its own Sys ID, and the Process Access Governance Revoke custom action that does the actual work.

Two-layer outputs, and a Draft that wasn't actually running. Inside the Custom Action's Script step, I wrote `outputs.result = JSON.stringify(result);` expecting that to be enough to surface the result. It wasn't. A Custom Action apparently needs a step-level output variable declared inside the Script step itself, separately mapped up to an Action-level output: writing to `outputs.result` alone just gets silently discarded with a warning buried in the execution log. On top of that, I lost some time testing against a version of the Action that looked saved but hadn't actually been published. Test can run against a stale draft without telling you, so a fix that looked like it wasn't working was actually just not being tested yet.

A choice list bug that only showed up with real data. The branching logic inside the Script Include checks which governance action was chosen and does the right thing for each. Every manual test I ran passed a typed string like "Revoke groups" and it worked perfectly, right up until a real submitted record flowed through the Flow for the first time, at which point the action came through as `revoke_roles_groups`, the field's actual internal value, not the label shown on the form. Every choice field in ServiceNow stores a separate internal name behind its display label, and I'd only ever tested against the label. The fix was straightforward once I saw it (check the internal values instead), but finding it meant learning to distrust my own manual tests, since typing a value by hand and having the platform hand you the real one are not the same test.

A List field that refused to travel through a Flow input. The Groups to Revoke field on the request table is a List field, capable of holding several group references at once. No matter which way I mapped it into the Custom Action's input, or which built-in transform I tried, it consistently arrived empty. After a lot of dead ends, I concluded this isn't a configuration mistake on my part: a List field's underlying value just isn't reliably exposed as a plain string through Flow Designer's standard input-mapping system. The fix was to stop fighting it: the Custom Action's script now fetches the field directly with its own `GlideRecord` query, keyed off the record's own ID, rather than depending on Flow Designer to hand it over as an input at all.

The deepest bug: a reference field pretending to be a string. Even after fixing the two bugs above, one input kept breaking every downstream operation that touched it: the target user's ID. It displayed correctly everywhere in Flow Designer's own execution log, looked like a normal `sys_id`, and yet the moment it reached my script, operations on it failed with an error saying a scoped record couldn't be cast to a scoped element. I added direct logging of the value's JavaScript type, and it came back as `object`, not `string`: the input was secretly still a full reference to the user record, not the plain ID it appeared to be. Reference-type fields, it turns out, need to be explicitly drilled down to their own Sys ID sub-field when mapped into a Flow input; mapping the reference itself, even into a field declared as a plain `String`, doesn't automatically resolve it the way it looks like it should. Fixing that one mapping, on every reference-type input the Action used, was what finally made the whole chain work end to end.

Process Access Governance Revoke

Process Access Governance Revoke
Revoked by Governance Engine
ba8e614dc397031026926Oct05013183

Action Outline < **Action Input** Create Input

Inputs

1 Script step

Error Evaluation

Outputs

Label	Name	Type	Mandatory	
Target User	userid	String	☒	
Group List	grouplist	String	☐	
Governance Action	action	String	☒	
Requested By	requestedby	String	☒	
Approved By	approvedby	String	☒	
Access Governance Id	accessgovernanceid	String	☒	

Figure 22: The Custom Action's declared inputs: Target User, Group List, Governance Action, Requested By, Approved By, and Access Governance Id, each typed as a plain String: the type every Reference-field pill still had to be drilled down to its Sys ID sub-field to actually satisfy.

1. Script step

Script

Input Variables

Name	Value	
userid	action ▶ Target User	
action	action ▶ Governance Action	
grouplist	action ▶ Group List	
requestedby	action ▶ Requested By	
approvedby	action ▶ Approved By	
accessgovernanceid	action ▶ Access Governance Id	

[Create Variable](#)

Script

```

1 (function execute(inputs, outputs) {
2   var reqGr = new GlideRecord('x_2037341_access_0_access_governance_request');
3   reqGr.get(inputs.accessgovernanceid + '');
4   var groupsListValue = reqGr.getValue('revoke_groups');
5
6
7   var processor = new AccessGovernanceRevokeProcessor();
8   var result = processor.processRequest(
9     inputs.userid + '',
10    inputs.action + '',
11    groupsListValue,

```

Figure 23: Inside the Script step: the input variables mapped in from the Action's inputs, and the first lines of the script, including the `reqGr.getValue('revoke_groups')` call that sidesteps Flow Designer's List-field mapping entirely by fetching the value directly, rather than trusting it to arrive as an input.

1. Script step

Script 

Script

```
1 (function execute(inputs, outputs) {  
2   var reqGr = new GlideRecord('x_2037341_access_0_access_governance_request');  
3   reqGr.get(inputs.accessgovernanceid + '');  
4   var groupsListValue = reqGr.getValue('revoke_groups');  
5  
6  
7   var processor = new AccessGovernanceRevokeProcessor();  
8   var result = processor.processRequest(  
9     inputs.userid + '',  
10    inputs.action + '',  
11    groupsListValue,  
12    inputs.requestedby + '',  
13    inputs.approvedby + '',  
14    inputs.accessgovernanceid + ''  
15  );  
16  outputs.result = JSON.stringify(result);  
17 })(inputs, outputs);
```

Output Variables

Label	Name	Type	Mandatory	
# result	result	String	☐	 

[+ Create Variable](#)

Figure 24: The complete script step: every input coerced with + '' before being passed into processRequest(), the call into AccessGovernanceRevokeProcessor, and the step-level result output variable that had to be declared here and separately mapped to the Action's own output before any value would surface outside the script.

Tools that made this debugging possible at all. None of this would have been findable without two things I only discovered partway through: Flow execution reporting is switched off by default on this instance, so execution records showed no useful detail whatsoever until I found and enabled the system property that turns on full configuration-and-runtime-value logging. And plain `gs.info()` calls, checked against the System Log, were what actually revealed the reference-field bug: the Flow execution UI on its own showed a value that looked completely correct, and only a raw type check exposed that it wasn't what it appeared to be.

7. Proof It Works: End to End

None of the individual pieces mattered much until they worked together, without me stepping in anywhere in the middle. The real test was this: submit a request through the actual Service Catalog form, approve it the way a real approver would, and change nothing else by hand (no background scripts, no manual triggers), and see whether the access change and the audit trail both appear on their own.

I ran this twice, once for each automated action, and both worked cleanly. For a Revoke groups request, selecting a real user and a real group they belonged to, submitting, and then marking the request Closed Complete resulted in that specific group membership disappearing from the user's record and a new, fully-populated row appearing in the audit table: target user, requester, approver, timestamp, and a plain-language description of exactly what was revoked, without me touching anything after clicking submit and later approving it.

For an Offboard user request, the same pattern held at a larger scale: every group the user belonged to was removed, every role they held directly was removed, their account was deactivated, and one audit row was written for every single group and role that came off: not one summary line, but an individually queryable record for each item, which is what actually makes an audit trail useful for a later compliance review rather than just a note that “something happened.”

All						
☐	Q	Number ▲	Priority	State	Assigned to	Task type
		ACC0001037	4 - Low	Closed Complete	(empty)	Access Governance Request
		ACC0001038	4 - Low	Closed Complete	(empty)	Access Governance Request
		ACC0001039	3 - Moderate	Open	Andrew Och	Access Governance Request

Figure 25: The real Access Governance Request queue during testing: ACC0001037 and ACC0001038 already Closed Complete from earlier runs, and ACC0001039 (the offboarding request for Abel Tutor, assigned to Andrew Och as approver), which is the request traced through the rest of this section once it's approved and closed.

EXECUTION DETAILS

Access Governance Request - Process Revoke Action

2

Process Access Governance Revoke

Completed

2026-09-09 08:11:26

747n

Configuration Details

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Target User	62826bf03710200044e0bfc8bcbe5df1	1 - Look Up Record ▶ ... ▶ Sys ID	String
Group List			String
Governance Action	offboard_user	1 ▶ ... ▶ Governance Action	String
Requested By	6816f79cc0a8016401c5a33be04be441	1 - Look Up Record ▶ ... ▶ Sys ID	String
Approved By	800b174138d089c868d09de320f9833b	1 - Look Up Record ▶ ... ▶ Sys ID	String
Access Governance Id	a02577d8c397c3102692b0cc05013166	Trigger - Record ... ▶ ... ▶ Sys ...	String

Output Data

VARIABLE NAME	RUNTIME VALUE	CONFIGURATION	TYPE
Action Status	{"Action Status":{"code":0,"message":"Success"}}	{"Action Status":{"code":{"\$cv":{"\$c":"java.lang.String","\$v":""}},"message":{"\$cv":{"\$c":"java.lang...	Object
Don't Treat as Error	true	true	True/False
result	{"status":"success","action":"offboard_user","result":{"status":"success","groupsRemoved":1,"directR...	result	String

No Logs

▶ Steps

Figure 26: The Flow execution detail for that exact request, showing Completed. Every configuration value resolved to something real: the target user, requester, and approver all arrived as genuine sys_ids via the Sys ID sub-field fix, and Governance Action arrived as its true internal value, offboard_user, not the display label. The output shows the actual JSON returned by processRequest(): status success, action offboard_user, and the group/role removal counts from offboardUser() itself.

Access Governance Id	Approved By	Date	Requested By	Revoke Name	Revoke Type	Short Description	Target User	Updated
ACC0001039	Andrew Och	2026-09-09 08:11:26	System Administrator	testGroup	Group	Revoked group testGroup from Abel Tuter	Abel Tuter	2026-09-09 08:11:26
ACC0001039	Andrew Och	2026-09-09 08:11:26	System Administrator	Conditional Script Writer	Group	Revoked group Conditional Script Writer ...	Abel Tuter	2026-09-09 08:11:26

Figure 27: Two of the real audit rows written for ACC0001039 (testGroup and Conditional Script Writer), both logged with revoke_type Group, approved_by Andrew Och, requested_by System Administrator, and target_user Abel Tuter, proof that the audit table gets one row per item actually affected, not a single bulk entry for the whole offboarding.

User ID: Email:

First name: Language:

Last name: Calendar integration:

Title: Time zone:

Department: Date format:

Password needs reset: ☐ Business phone:

Locked out: ☒ Mobile phone:

Active: ☐ Photo: [Click to add...](#)

Identity type:

Internal Integration User: ☐

[Update](#) [Set Password](#) [Delete](#)

Related Links

[View linked accounts](#)

[View Subscriptions](#)

[Reset a password](#)

[Roles](#) [Groups](#) [Delegates](#) [User Client Certificates](#)

Role: Search:

Figure 28: Abel Tuter's user record after the offboarding completed: Active unchecked and Locked out checked; that second box isn't something the application sets directly, it's the platform's own "Lock Out User" business rule reacting automatically to the deactivation, exactly the kind of independent platform behavior worth testing around rather than assuming. The Roles list beneath is empty, consistent with both the direct roles Piece B deleted explicitly and the group-inherited roles that disappeared on their own once Piece A removed their groups.

One platform behavior worth calling out, because I didn't build it and it matters a lot to how this system actually behaves: removing a group membership on this platform automatically cascades and cleans up any role a user held only because of that group, without any extra code on my part. I confirmed this directly: in one test, revoking a single group correctly removed 44 dependent role rows that existed purely because one role that group granted itself contained many others. The application relies on that platform behavior rather than reimplementing it, which is also why the audit logic distinguishes between what the application directly revoked and what the platform cleaned up as a consequence.